SIC Project Final Report

Title

AI Research Agent (RAG): LLM-powered tool for querying private document PDF sets

Project ID

SIC/AI/013

Team Members

- **Member 1:** Siddharth Sinha – Lead

Abstract

This project presents an LLM-powered Retrieval-Augmented Generation (RAG) system for querying private PDF document collections. The system is designed to ingest PDF documents, preprocess and chunk the extracted text, generate embeddings using a custom Keras-based Siamese model, retrieve relevant passages through hybrid dense and lexical search, and generate grounded answers using a lightweight local LLM through Ollama. The project combines classical Natural Language Processing (NLP) techniques with modern retrieval and generation methods to build a private document question-answering system. The final system supports both local LLM-based answer generation and extractive fallback answering when generation is unavailable.

---

1. Introduction

With the increasing volume of unstructured documents, retrieving useful information from private PDFs has become an important problem in AI and data systems. Traditional keyword search often fails to capture semantic meaning, while general-purpose LLMs may hallucinate when not grounded in source documents. Retrieval-Augmented Generation (RAG) addresses this by first retrieving relevant information and then generating answers using only that retrieved context.

This project aims to build a private document QA system that can answer natural-language questions from PDF files. The system is designed to work locally, maintain document grounding, and support lightweight LLM inference using Ollama.

---

2. Problem Statement

Organizations and students often store useful information in PDF files such as manuals, reports, technical guides, and notes. However, searching through these documents manually is slow and inefficient. The problem addressed in this project is:

How can we build an intelligent system that can read a private collection of PDF documents, retrieve the most relevant information, and answer user questions accurately and efficiently?

---

3. Objectives

The main objectives of the project are:

- To ingest and process private PDF documents
- To preprocess document text using NLP techniques
- To split documents into meaningful chunks for retrieval
- To build a semantic embedding model using Keras
- To perform hybrid retrieval using dense vectors and lexical matching
- To generate grounded answers using a lightweight local LLM
- To provide a complete end-to-end private document question-answering pipeline

---

4. Methodology Overview

The project follows a multi-stage pipeline:

1. **PDF Extraction**
2. **NLP Preprocessing**
3. **Chunking**
4. **Embedding Generation**
5. **Vector Store Construction**
6. **Hybrid Retrieval**
7. **LLM-based Answer Generation**
8. **Extractive Fallback Answering**

The system is implemented in Python and combines classical NLP with neural embedding and local LLM generation.

---

5. Dataset Description

The project uses PDF documents as the primary dataset. These PDFs are placed in the data/pdfs/ directory and serve as the knowledge base for the system.

Dataset Sources

The system was tested using PDF files such as:

- apigateway-dg.pdf
- rag_quality_test.pdf
- test_document.pdf

Raw Data

The raw data consists of PDF documents stored in the data/pdfs/ folder.

Processed Data

After ingestion, processed data is stored in:

- data/processed/chunks.json
- data/vectors/

The processed chunk file contains:

- raw chunk text
- processed chunk text
- source file metadata
- page information
- chunk identifiers

---

6. NLP Preprocessing

The project uses NLTK for preprocessing and normalization.

Steps Used

- Sentence tokenization
- Word tokenization
- Lowercasing
- Stopword removal
- Part-of-speech tagging
- Lemmatization

Explanation

Tokenization is the process of splitting text into smaller units such as sentences or words.

Stopword removal removes very common words such as “the”, “is”, and “and” that often add little meaning to retrieval.

POS tagging identifies the grammatical role of each word, such as noun, verb, adjective, or adverb.

Lemmatization converts words into their base or dictionary form. For example:

- running -> run
- cars -> car
- studies -> study

This improves retrieval by normalizing different forms of the same word.

---

7. Chunking Strategy

After preprocessing, documents are split into smaller pieces called chunks. Chunking is necessary because:

- full documents are too large for retrieval and generation
- smaller chunks improve search precision
- chunk overlap preserves context across boundaries

Chunking Approaches

The system supports:

- **Fixed chunking**
- **Sentence-aware chunking**

In the final version, sentence-aware chunking was used because it preserves natural sentence boundaries and improves answer readability.

---

8. Model Design

The project uses a custom **Siamese neural network** built with TensorFlow/Keras for semantic embedding generation.

Siamese Model

A Siamese model takes two text inputs and learns whether they are semantically similar. This helps create an embedding space where related chunks are closer together.

Supported Encoder Architectures

- BiLSTM
- BiGRU
- CNN

The final system primarily used the **BiLSTM-based encoder**.

Why This Model Was Used

- It supports semantic similarity learning
- It is lightweight compared to large transformer-based retrievers
- It fits the project's goal of combining classical NLP and custom learning

---

9. Training Process

The training pipeline works as follows:

1. Load processed chunks
2. Build a vocabulary
3. Create positive and negative chunk pairs
4. Encode chunk text as sequences
5. Train the Siamese embedding model
6. Save the trained encoder
7. Rebuild the vector store using the trained model

Model Output

The trained model is saved in:

- models/siamese_bilstm/encoder.keras

Supporting artifacts such as vocabulary are also stored in the model directory.

---

10. Retrieval System

The retrieval system is hybrid, meaning it combines:

1. Dense Retrieval

Dense retrieval uses embedding vectors produced by the trained model. Query vectors are compared with chunk vectors using similarity measures.

2. Lexical Retrieval

Lexical retrieval uses TF-IDF to match exact or near-exact query terms with chunks.

3. Reranking

The retrieved results are reranked based on:

- dense similarity
- lexical similarity
- query overlap
- penalties for poor chunk patterns such as headings or overly short chunks

Benefit of Hybrid Retrieval

Hybrid retrieval improves performance by combining:

- semantic understanding from embeddings
- exact keyword matching from TF-IDF

This is especially useful for technical documents.

---

11. Answer Generation

The final project uses **LLM-powered answer generation**.

Local LLM Through Ollama

A lightweight local model is used through Ollama, for example:

- phi3:mini

Answering Process

1. Retrieve top chunks
2. Compress context to fit the model context window
3. Send only retrieved evidence to the LLM
4. Generate a final grounded answer

Extractive Fallback

If the LLM is unavailable, the system falls back to extractive answering:

- relevant snippets are selected from retrieved chunks
- duplicates are removed
- the answer is returned directly from source text

This ensures the system remains usable even without generation.

---

12. Tools and Technologies Used

- **Python** for implementation
- **NLTK** for preprocessing
- **PyMuPDF** for PDF extraction
- **TensorFlow / Keras** for embedding model training
- **scikit-learn** for TF-IDF and lexical retrieval
- **NumPy** for vector operations
- **Ollama** for local LLM inference
- **Jupyter Notebook** for experimentation and EDA

---

13. Baseline Results

The initial baseline version of the project used:

- preprocessing
- chunking

- TF-IDF fallback or simple retrieval
- extractive question answering

Baseline Behavior

- retrieval worked but was less precise on large documents
- answers were often repetitive or heading-heavy
- definition-style questions sometimes returned noisy snippets

Even though baseline quality was limited, it successfully demonstrated the end-to-end pipeline.

---

14. Optimization and Final Improvements

The following optimizations were made during the project:

- separated raw text and processed text
- improved chunking strategy
- added sentence-aware chunking
- improved training compatibility for small corpora
- fixed model loading issues
- added hybrid dense + lexical retrieval
- improved reranking logic
- improved extractive answer formatting
- added LLM-based generation through Ollama
- updated documentation and notebook support

Final Outcome

The final system:

- ingests PDFs successfully
- trains a custom embedding model
- retrieves relevant chunks using hybrid retrieval
- answers using a lightweight local LLM
- supports extractive fallback mode
- works on multiple document sets

---

15. Evaluation and Results

The system was evaluated through practical query testing on ingested documents.

Sample Result Types

- Definition questions
- Technical questions from manuals
- Questions from smaller test PDFs
- Multi-document retrieval scenarios

Observed Final Performance

- retrieval became more stable after hybrid retrieval was added
- answer quality improved significantly after LLM integration
- local LLM generation provided more natural answers than extractive-only mode
- very large technical PDFs remained more challenging than small focused documents

Confusion Matrix

A confusion matrix is **not applicable** in the traditional sense because this project is not a standard classification problem. It is a document retrieval and question-answering system.

---

16. Challenges Faced

Some major challenges during the project were:

- handling noisy extraction from PDFs
- selecting the right chunk size and overlap
- dealing with repetitive or heading-heavy technical manual text
- avoiding irrelevant retrieval from large documents
- making answer generation grounded rather than hallucinated
- integrating lightweight local LLM generation effectively

These challenges were addressed through iterative improvements in chunking, retrieval, reranking, and answer generation.

---

17. Advantages of the System

- Works on private documents
- Supports local LLM generation
- Provides grounded answers from source material
- Combines semantic and lexical retrieval
- Modular design for future improvements
- Supports experimentation with different encoder models

---

18. Limitations

Despite its success, the project has some limitations:

- answer quality depends on retrieval quality
- the local LLM is smaller and less powerful than large cloud-based models
- very large technical PDFs can still produce imperfect retrieval
- the Siamese embedding model learns similarity, not general reasoning
- chunking quality strongly influences final answers

---

19. Future Scope

Possible future improvements include:

- using stronger reranking models
- adding citation highlighting
- improving PDF layout handling
- using better local LLMs
- adding a web-based user interface
- introducing feedback-based retrieval tuning
- supporting more file formats such as DOCX and TXT

---

20. Conclusion

This project successfully developed an **LLM-powered RAG system for private PDF question answering**. It combines PDF extraction, NLTK-based preprocessing, chunking, Keras-based semantic embeddings, hybrid retrieval, and local LLM generation using Ollama. The final system can answer natural-language questions from private documents in a grounded and practical way.

The project demonstrates how classical NLP, neural embeddings, and local LLMs can be combined to build a useful real-world AI application for private knowledge retrieval.

---

21. References

You can include references like:

1. NLTK Documentation
2. TensorFlow / Keras Documentation
3. scikit-learn Documentation
4. Ollama Documentation
5. PyMuPDF Documentation
6. Research papers or articles on Retrieval-Augmented Generation
7. Research papers on Siamese networks for semantic similarity